



UNITED INTERNATIONAL UNIVERSITY (UIU)
Dept. of Computer Science & Engineering
Trimester: Summer 2026 (Class Test-1)
Course No: CSE 3812 Title: Artificial Intelligence Laboratory
Section: H Time: 45 mins Marks: 30

Instructions:

- While answering the questions make sure to follow this convention,
 - First create a new folder in any directory on your current PC where you want to write your code. The folder name will be your rollNumber_pythonTest e.g. for me, it will be 1907016_pythonTest
 - Then go into the folder. The folder will contain both the python scripts as answers to the two questions provided.
 - Suppose for me, my roll was 1907016. So, the first filename should be 1907016_a.ipynb and the second one should be 1907016_b.ipynb. For you, only the roll number part will change. [If someone wants to write in a (dot)py script, that's completely fine too]
 - Properly save the scripts at regular intervals if you're coding on an offline basis because unlike google colab, offline codes sometimes don't have autosave option on.
 - If you want to code online in Google colab, then you should create a new notebook after going into <https://colab.research.google.com/> following the same name convention.
 - After you're done with the coding part, you should download the notebook from the File option on the top left corner as an ipynb file in the folder directory that I instructed as before.
- If you followed the instructions properly, now your folder directory has two ipynb/py files in it.
- Then, upload the whole folder in this [Link](#).

MCQ: <https://forms.gle/1eo7FydytTXqWnKBA>

The Planetary Security Guard

At the edge of the galaxy, the planetary outpost Aegis-9 maintains a strict defensive grid. The sky is monitored using a two-dimensional coordinate system with integer coordinates $(\mathbf{x}, \mathbf{y})$. To manage incoming space traffic, the defense authority has established **concentric defensive grids** and uses authorization codes $\mathbf{A}$ (which are positive integers) to verify flight permits.

The defensive airspace is classified into three distinct zones based on the coordinates $(\mathbf{x}, \mathbf{y})$ of an approaching spaceship:

- **Inner Core (Red Zone):** Coordinates that strictly lie inside the boundaries: $15 < \mathbf{x} < 45$ and $25 < \mathbf{y} < 55$.
- **Outer Perimeter (Yellow Zone):** Coordinates that lie within $10 \leq \mathbf{x} \leq 50$ and $20 \leq \mathbf{y} \leq 60$, but are *not* inside the Inner Core.
- **Deep Space (Green Zone):** Any coordinate that lies entirely outside the Outer Perimeter boundaries.

Write an automated security protocol to analyze the ship's coordinates and authorization code **A**:

- If the ship is in the **Inner Core (Red Zone)**:
 - It is permitted if its authorization code A is strictly greater than 100 and is also evenly divisible by 7 (i.e., $A \% 7 == 0$). If both conditions are met, print "AUTHORIZED CORE ACCESS".
 - Otherwise, print "SECURITY ALERT: CORE VIOLATION".
- If the ship is in the **Outer Perimeter (Yellow Zone)**:
 - It is permitted if its authorization code A is an even number (i.e., $A \% 2 == 0$) OR if the coordinate is perfectly diagonal (i.e., $X == Y$). If either condition is met, print "PERMITTED PERIMETER PASS".
 - Otherwise, print "WARNING: PERIMETER CHECK REQUIRED".
- If the ship is in **Deep Space (Green Zone)**:
 - Print "SAFE: FREE NAVIGATION".

Input and Output Specifications

Specification	Details
Input Format	- The first line of input is an integer, X, representing the horizontal coordinate of the ship. - The second line of input is an integer, Y, representing the vertical coordinate of the ship. - The third line of input is an integer, A, representing the ship's authorization code.
Output Format	- Print a single string representing the outpost security protocol response.

Test Case Scenarios

Sample Input	Sample Output	Conceptual Verification
30 40 140	AUTHORIZED CORE ACCESS	- Coordinate (30, 40) is strictly in the Inner Core ($15 < 30 < 45$ and $25 < 40 < 55$). - Authorization code 140 is > 100 and divisible by 7 ($140 \% 7 == 0$). Core access is authorized.
12 22 15	WARNING: PERIMETER CHECK REQUIRED	- Coordinate (12, 22) is not in the Inner Core but is in the Outer Perimeter ($10 \leq 12 \leq 50$ and $20 \leq 22 \leq 60$). - Authorization code 15 is not even, and $X \neq Y$ ($12 \neq 22$). Manual perimeter check is required.
12 12 15	SAFE: FREE NAVIGATION	- Coordinate (12, 12) is outside the Outer Perimeter ($Y = 12$ is less than 20). The zone is safe for free navigation.

Evaluation Rubric

Assessment Component	Target Performance Criteria	Score Weight
Input Extraction & Casting	Correctly parses and converts input values into the expected types (X, Y, and A to integers).	2.0 Points
Inner Core Logic	Correctly implements coordinate bounds and nested authorization logic ($A > 100$ and $A \% 7 == 0$).	2.0 Points
Outer Perimeter Logic	Correctly implements coordinate bounds for the outer range, ensuring Inner Core coordinates are excluded.	2.0 Points
Perimeter Authorization Logic	Correctly implements compound logic checking if authorization code is even or coordinates are diagonal ($X == Y$).	2.0 Points
Conditional Structure & Style	Correctly organizes logical blocks using appropriate conditional structures (if-elif-else) and outputs exact strings.	2.0 Points

The Interstellar Invoice and Taxation System

At the Edge of the Universe trading outpost, merchants use a dynamic pricing, discount, and tax calculator to process invoices for the **Galactic Guild of Explorers**. To reward guild loyalty, purchase discounts are determined based on the buyer's **guild membership tier**, their **raw purchase amount P** in credits, and a **dynamic local planetary tax rate T** (expressed as a percentage).

Write a program that processes the invoice in three stages:

1. **Read and Parse Inputs:** Extract the membership tier (string), the raw purchase amount P (float), and the planetary tax rate T (float).
2. **Calculate the Discounted Price (D):** Use the following tier and conditional rules:
 - **Gold Tier:**
 - An initial 20% discount is applied to the raw purchase amount P .
 - If this new discounted price is strictly greater than 400.0 credits, apply an additional flat discount of 50.0 credits.
 - Otherwise, if the integer value of the original purchase is even (i.e., $\text{int}(P) \% 2 == 0$), deduct an additional flat 10.0 credits instead.
 - **Silver Tier:**
 - If the raw purchase P is strictly greater than 300.0 credits, apply a 15% discount.
 - If raw P is between 100.0 and 300.0 credits (inclusive), apply a 10% discount.
 - Otherwise, apply a flat 5.0 credits discount, ensuring the discounted price does not drop below 0.0 credits (use conditional logic to prevent negative prices).
 - **Bronze Tier:**
 - If raw P is strictly greater than 150.0 credits, check the integer part of the raw purchase. If the integer part of the purchase is even (i.e., $\text{int}(P) \% 2 == 0$), apply a total discount of 7%. If the integer part is odd, apply a total discount of 5%.
 - If $P \leq 150.0$ credits, no discount is applied.
 - **No Tier (or any other string):** No discount is applied.
3. **Apply Local Tax:** Once the **final discounted price D** is computed, apply the **planetary tax rate T** to find the final total using the equation:

$$Total = D + D \times (T/100.0)$$

Input and Output Specifications

Specification	Details
Input Format	- The first line of input is a string representing the membership tier ("Gold", "Silver", "Bronze", or "None"). - The second line of input is a float, P, representing the raw purchase amount ($P \geq 0.0$). - The third line of input is a float, T, representing the planetary tax percentage ($T \geq 0.0$).
Output Format	- Print a single floating-point number representing the final discounted and taxed transaction price, formatted to exactly two decimal places.

Test Case Scenarios

Sample Input	Sample Output	Conceptual Verification
Gold 600.0 5.0	451.50	- Gold membership applies a 20% discount: $600.0 * 0.8 = 480.0$. - Since $480.0 > 400.0$, an additional 50.0 flat discount is applied: 430.0. - Applying a 5% tax rate: $430.0 * 1.05 = 451.50$.
Gold 350.0 10.0	297.00	- Gold membership applies a 20% discount: $350.0 * 0.8 = 280.0$. - Since 280.0 is not > 400.0, check the original price: $\text{int}(350.0) \% 2 == 0$ is True, so a 10.0 flat discount is applied: 270.0. - Applying a 10% tax rate: $270.0 * 1.1 = 297.00$.
Silver 80.0 8.5	81.38	- Silver membership under 100.0 applies a 5.0 flat discount: $80.0 - 5.0 = 75.0$. - Applying an 8.5% tax rate: $75.0 * 1.085 = 81.375$, which formats to 81.38.

Bronze	197.16	- Bronze membership with raw purchase > 150.0. Since $\text{int}(200.0) \% 2 == 0$ is True, apply a 7% discount: $200.0 * 0.93 = 186.0$.
200.0		
6.0		- Applying a 6% tax rate: $186.0 * 1.06 = 197.16$.

Evaluation Rubric

Assessment Component	Target Performance Criteria	Score Weight
Input Extraction & Casting	Correctly parses and converts all user inputs (membership string, float price, and float tax rate).	2.0 Points
Primary Tier Routing	Separates logical paths correctly for Gold, Silver, Bronze, and None tiers.	3.0 Points
Sub-Conditional Logic	Correctly applies sub-conditions within tiers (Gold's price threshold or int checking; Silver's multiple bounds and floor limit; Bronze's nested int-even check).	3.0 Points
Taxation, Math, & Formatting	Accurately computes the final total with tax applied, and formats the output to exactly two decimal places.	2.0 Points

